

EL KATTOUFI Youssef

Année 2026 – 2027

TP 1 IOT



Etablissement scolaire : Polytech Sorbonne

Formation : Electronique et Informatique

Matières : IOT

Enseignant : Yan DOUZE

Table des matières

Remerciements.....	4
Résumé	5
I. Objectifs du TP.....	6
II. Démarche suivie	7
III. Explication brève de la configuration de l’http REST	8
A. Code ESP32	9
B. Node-RED.....	9
IV. Explication brève de la configuration du broker	11
A. Code ESP32	12
B. Node-RED.....	13
V. Résultat.....	14
VI. Sources.....	14

Remerciements

Je tiens tout d'abord à remercier chaleureusement Monsieur Yan Douze qui nous a encadré tout au long de ce TP. Son accompagnement, ses conseils et ses enseignements m'a permis de mener à bien ce TP, tout en nous apportant le socle de connaissances nécessaires en IOT.

Résumé

Au sein de la formation d'ingénieur en Électronique et Informatique à Polytech Sorbonne, nous avons suivi l'enseignement Internet des Objets (IoT), qui nous a permis d'acquérir des connaissances fondamentales dans ce domaine. Cette matière nous a notamment introduits à la communication entre objets connectés, à l'utilisation de microcontrôleurs tels que l'ESP32, ainsi qu'aux différents protocoles de communication utilisés dans les systèmes IoT.

Cet enseignement m'a permis de mieux comprendre le fonctionnement global d'un système connecté, depuis l'acquisition des données par des capteurs jusqu'à leur transmission, leur traitement et leur visualisation. Il met également en évidence l'importance de la communication réseau, de la gestion des données et de l'interconnexion entre les différents équipements d'un système IoT.

I. Objectifs du TP

Dans le cadre de ce TP, j'ai choisi de travailler sur mon ordinateur personnel, car je suis plus à l'aise sur cet environnement et j'en maîtrise entièrement la configuration. Cela m'a permis de travailler plus efficacement, de mieux comprendre les éventuels problèmes de compilation ou de dépendances et de garder la maîtrise de l'ensemble de ma chaîne de développement.

J'ai utilisé Visual Studio Code avec l'extension PlatformIO (car je trouve que c'est mieux par rapport à Arduino IDE).

PlatformIO permet de développer et de téléverser des programmes sur des cartes comme l'ESP32 avec le framework Arduino. Le principe reste donc très proche de l'Arduino IDE : on écrit le programme, on le compile puis on le téléverse sur la carte. La différence principale est que PlatformIO s'intègre à VS Code et apporte une gestion de projet plus structurée, notamment pour les bibliothèques, les dépendances, les différentes cartes et les paramètres de compilation.

Je préfère également cette méthode car elle me laisse davantage de contrôle sur le projet. En particulier, il est possible d'ajouter des bibliothèques locales dans le dossier `lib/`, d'utiliser une version précise d'une dépendance ou, si nécessaire, de travailler sur une version modifiée d'une bibliothèque pour l'adapter au besoin du projet.

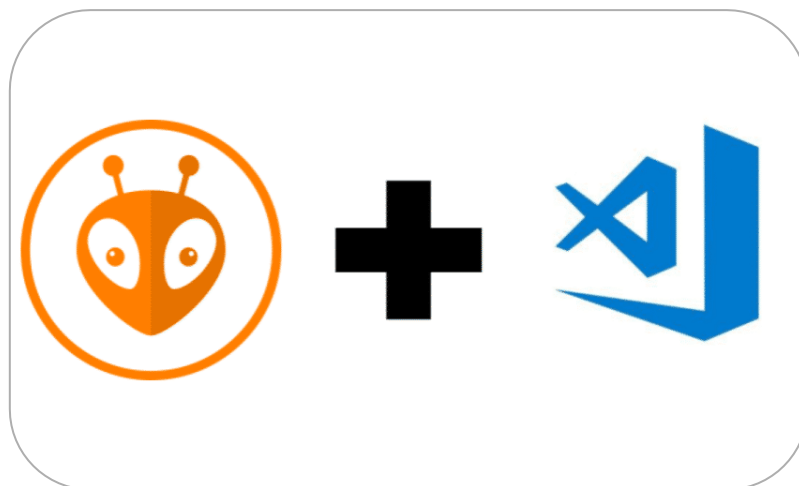


Figure 1 : Visualisation Platform IO + Visual Studio Code

Remarque :

Comme à mon habitude le projet se trouve sur mon compte github personnel à l'adresse suivante : https://github.com/Darkzvellier/TP_1_IOT_S7_polytech_sorbonne

II. Démarche suivie

La réalisation du TP a été effectuée progressivement afin de tester chaque partie du système indépendamment avant de les regrouper.

Le dossier Esp32_DHT_11 correspond à la première étape du TP. Il permet de vérifier le fonctionnement du capteur DHT11 et de lire localement la température et l'humidité sur le moniteur série.

Le dossier Esp32_to_Node_red permet de tester la communication HTTP POST entre l'ESP32 et Node-RED à l'aide d'un simple compteur, afin de valider la connexion Wi-Fi et les échanges réseau.

Le dossier Esp32_to_Node_red_et_DHT11 correspond à la version REST complète. L'ESP32 envoie à Node-RED le compteur, la température et l'humidité, puis récupère l'état de la LED à l'aide d'une requête GET.

Le dossier Esp32_to_Mosquitto_et_DHT11 contient la version MQTT. L'ESP32 publie les mesures du DHT11 vers Mosquitto et reçoit les commandes permettant de contrôler la LED.

Enfin, le dossier Node_red_architecture contient le flow Node-RED utilisé pour les échanges, le traitement et la visualisation des données.

Voici donc l'arborescence générale du projet :

```

TP1/
|
|-- README.md
|-- CR_IOT_Youssef_EL_KATTOUFI.docx
|
|-- Esp32_DHT_11/
|   |-- platformio.ini
|   |-- src/
|   |   |-- main.cpp
|   |-- include/
|   |-- lib/
|   |-- test/
|
|-- Esp32_to_Node_red/
|   |-- platformio.ini
|   |-- src/
|   |   |-- main.cpp
|   |-- include/
|   |-- lib/
|   |-- test/
|
|-- Esp32_to_Node_red_et_DHT11/
|   |-- platformio.ini
|   |-- src/
|   |   |-- main.cpp
|   |-- include/
|   |-- lib/
|   |-- test/
|
|-- Esp32_to_Mosquitto_et_DHT11/
|   |-- platformio.ini
|   |-- src/
|   |   |-- main.cpp
|   |-- include/
|   |-- lib/
|   |-- test/
|
|-- Node_red_architecture/
|   |-- flow_cpt_hum_temp_dashboard.json
|
|-- DHT-11_datasheet.pdf
|-- TP1_architecture_REST-MQTT_2026.pdf
|-- Tutorial_Getting_Started_ESP32.pdf
|-- Node_red.url
|-- NodeRED_Manager.bat - Raccourci.lnk
|-- git_push.bat
|-- .gitignore

```

Figure 2 : Arborescence générale du répertoire

III. Explication brève de la configuration de l'http REST

Voici l'architecture employé cette partie du tp:

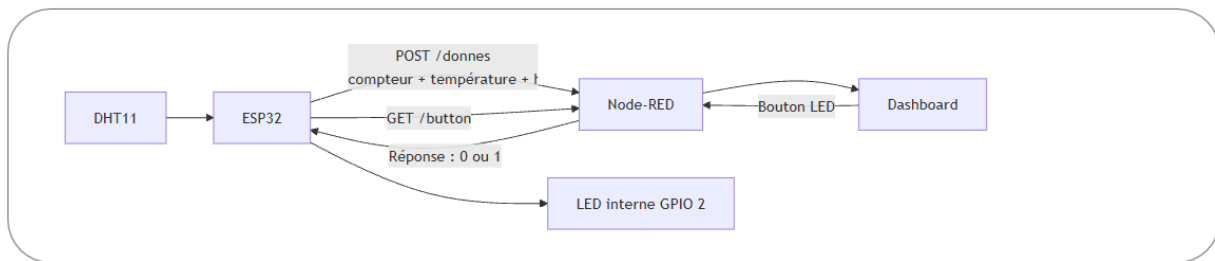


Figure 3 : Architecture http REST

A. Code ESP32

L'ESP32 commence par se connecter au réseau Wi-Fi à l'aide de la fonction `WiFi.begin(ssid, password)`. Une fois la connexion établie, il peut communiquer avec Node-RED en utilisant le protocole HTTP.

Deux routes sont utilisées dans Node-RED. La route `/donnes` permet à l'ESP32 d'envoyer les différentes mesures à l'aide d'une requête **HTTP POST**, tandis que la route `/button` permet de récupérer l'état souhaité de la LED avec une requête **HTTP GET**.

Pour envoyer les données, l'ESP32 utilise la bibliothèque `HTTPClient`. Le format des données est précisé avec l'en-tête `Content-Type: application/x-www-form-urlencoded`. Les valeurs du compteur, de la température et de l'humidité sont ensuite regroupées sous la forme `compteur=...&hum=...&temp=...`, puis envoyées à Node-RED avec la fonction `httpPOST.POST(...)`.

Dans l'autre sens, l'ESP32 effectue régulièrement une requête GET sur la route `/button`. Node-RED renvoie alors une valeur "1" ou "0". La valeur "1" permet d'allumer la LED de l'ESP32, tandis que la valeur "0" permet de l'éteindre.

Enfin, les codes de réponse HTTP ainsi que les données envoyées et reçues sont affichés dans le moniteur série. Cela permet de vérifier que la communication entre l'ESP32 et Node-RED fonctionne correctement.

Une correction est également nécessaire concernant la lecture du capteur DHT11. Dans le code, les valeurs de température et d'humidité sont inversées. La température doit être récupérée avec `dht.readTemperature()` et l'humidité avec `dht.readHumidity()`.

B. Node-RED

Dans cette partie, Node-RED est utilisé comme serveur REST afin de communiquer avec l'ESP32. Une première route HTTP POST `/donnes` permet de recevoir les données envoyées par la carte.

Les données reçues contiennent la température, l'humidité ainsi que la valeur du compteur. Elles sont ensuite séparées à l'aide de plusieurs nœuds fonction. Chaque valeur est envoyée vers l'affichage correspondant dans le dashboard Node-RED, notamment Temperature, Humiditer et Compteur.

Des nœuds debug sont également présents dans le flow. Ils permettent d'observer directement les données reçues et de faciliter la recherche d'erreurs pendant les différents tests. Une fois les données traitées, le nœud response renvoie une réponse HTTP à l'ESP32 afin de terminer correctement la requête POST.

Pour commander la LED, un switch est présent dans le dashboard Node-RED. L'état sélectionné par l'utilisateur est traité par un nœud fonction, puis conservé afin que l'ESP32 puisse venir le récupérer.

L'ESP32 effectue alors une requête HTTP GET sur la route /button. Node-RED lui renvoie l'état actuel de la LED sous la forme d'une valeur 1 ou 0. La valeur 1 correspond à l'allumage de la LED, tandis que la valeur 0 correspond à son extinction. Le nœud Réponse LED permet de terminer la requête HTTP et de transmettre cette valeur à l'ESP32.

Ainsi, Node-RED joue ici le rôle de serveur REST. Il reçoit les différentes mesures de l'ESP32 avec la requête POST /donnees et transmet en retour l'état de la LED grâce à la requête GET /button.

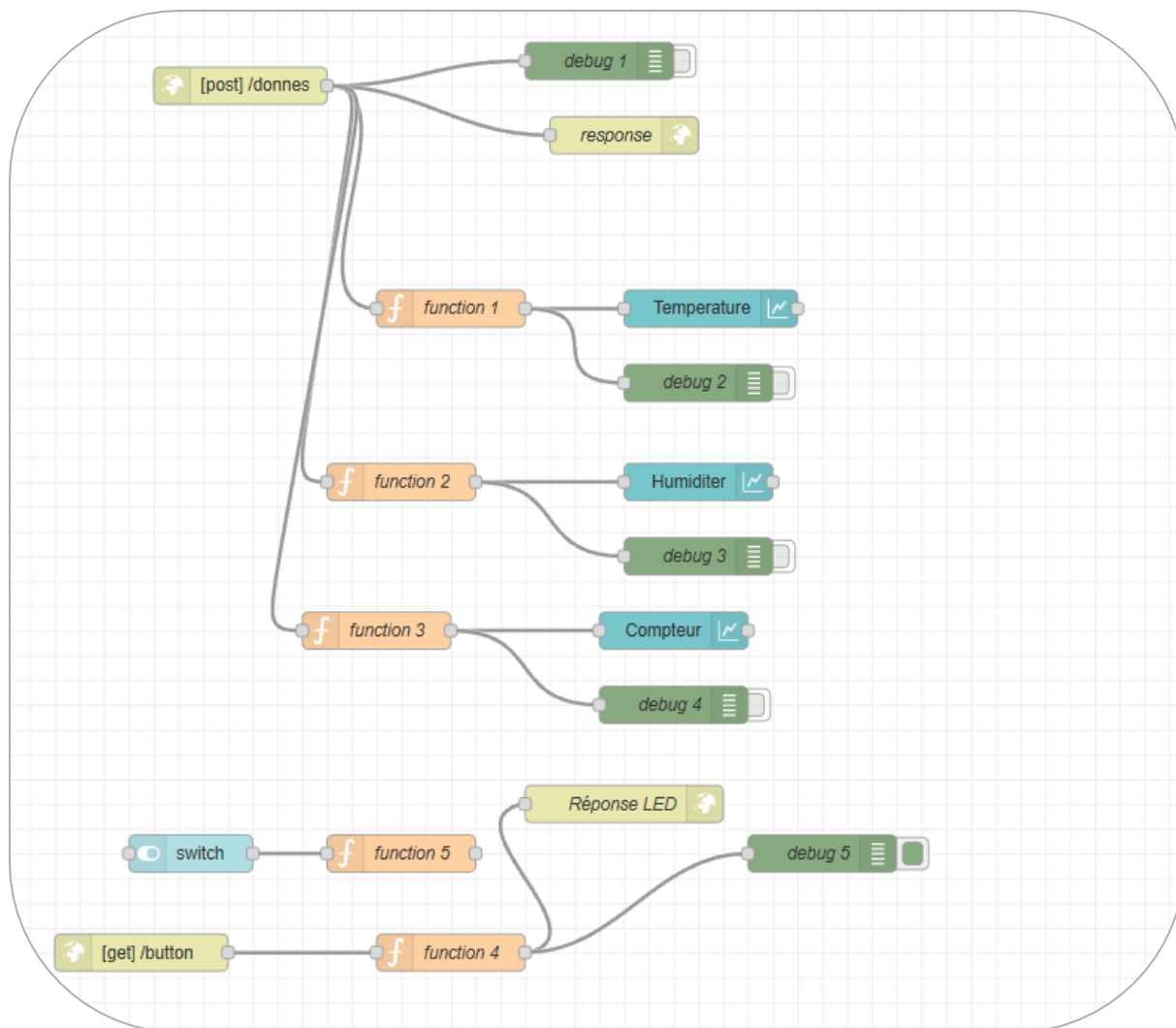


Figure 4 : Flow REST

Les fonctions 5 et 6 permettent de faire le lien entre le switch du dashboard et l'ESP32. La première convertit et mémorise l'état du switch dans une variable flow, tandis que la seconde récupère cette valeur lors de la requête GET afin de la transmettre à l'ESP32.

IV. Explication brève de la configuration du broker

Voici l'architecture employé sur cette part mosquitto :

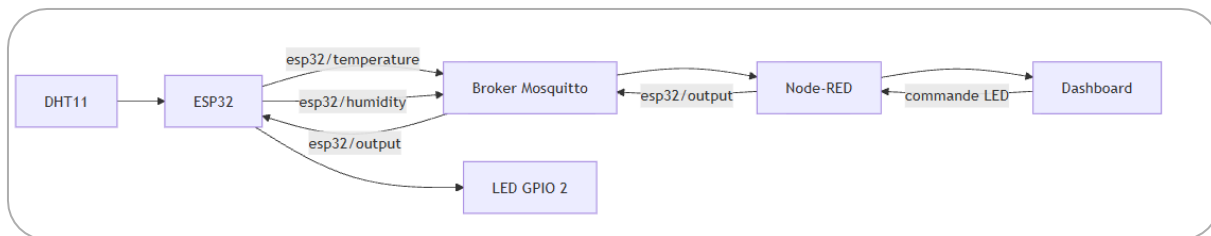


Figure 5 : Architecture MQTT

A. Code ESP32

Dans cette partie, l'ESP32 commence par se connecter au réseau Wi-Fi, puis au broker MQTT Mosquitto à l'aide de l'adresse IP du PC et du port 1883. La bibliothèque `PubSubClient` est utilisée pour gérer l'ensemble des échanges MQTT entre l'ESP32 et le broker.

Dans la fonction `setup()`, le serveur MQTT est configuré avec `client.setServer(mqtt_server, 1883)`. La fonction `client.setCallback(callback)` permet également d'indiquer quelle fonction devra être exécutée lorsqu'un nouveau message MQTT est reçu par l'ESP32.

Dans la boucle principale, le programme vérifie régulièrement que l'ESP32 est toujours connecté au broker Mosquitto. Si la connexion est perdue, la fonction `reconnect()` tente automatiquement de la rétablir. Une fois connecté, `client.loop()` permet de maintenir la communication MQTT et de traiter les éventuels messages reçus.

Environ toutes les 500 ms, l'ESP32 effectue une nouvelle lecture du capteur DHT11. La température est ensuite publiée sur le topic `esp32/temperature`, tandis que l'humidité est publiée sur le topic `esp32/humidity`. Avant l'envoi, les valeurs numériques sont converties en chaînes de caractères à l'aide de la fonction `dtostrf()`, puis transmises au broker avec `client.publish()`.

Dans l'autre sens, l'ESP32 s'abonne au topic `esp32/output` grâce à la fonction `client.subscribe("esp32/output")`. Ce topic permet notamment de recevoir une commande provenant de Node-RED afin de modifier l'état de la LED présente sur l'ESP32.

La fonction `callback()` est appelée automatiquement lorsqu'un nouveau message est reçu sur un topic auquel l'ESP32 est abonné. Elle récupère le nom du topic concerné, le contenu du message ainsi que sa taille. Comme le message est reçu sous forme de tableau d'octets, il est d'abord reconstruit dans une chaîne de caractères afin de pouvoir être comparé facilement.

Le programme vérifie ensuite que le message reçu provient bien du topic `esp32/output`. Si le contenu du message est `"true"`, la LED est allumée avec `digitalWrite(LED_PIN, HIGH)`. À

l'inverse, si le message reçu est "false", la LED est éteinte avec `digitalWrite(LED_PIN, LOW)`.

Enfin, la fonction `reconnect()` permet de rétablir automatiquement la connexion au broker Mosquitto en cas de coupure. L'ESP32 tente alors de se reconnecter avec l'identifiant `ESP32Client_Youssef`. Dès que la connexion est de nouveau établie, il se réabonne au topic `esp32/output` afin de pouvoir continuer à recevoir les commandes destinées à la LED.

B. Node-RED

Dans cette partie, Node-RED est configuré pour communiquer avec l'ESP32 en utilisant le protocole MQTT. Le broker utilisé est Mosquitto, qui est lancé directement sur le PC. Dans la configuration des nœuds MQTT de Node-RED, l'adresse IP du PC est donc renseignée puisque c'est lui qui héberge le broker.

L'ESP32 utilise cette même adresse IP pour se connecter à Mosquitto, avec par exemple `const char *mqtt_server = "192.168.109.171"`. La communication MQTT utilise le port standard 1883. Ainsi, Node-RED et l'ESP32 communiquent tous les deux avec le même broker Mosquitto hébergé sur le PC.

Pour récupérer les mesures provenant de l'ESP32, deux nœuds MQTT In sont utilisés dans Node-RED. Le premier est abonné au topic `esp32/temperature` afin de recevoir la température, tandis que le second est abonné au topic `esp32/humidity` pour récupérer l'humidité.

Dès que l'ESP32 publie une nouvelle mesure, Mosquitto transmet automatiquement cette valeur aux nœuds Node-RED abonnés aux topics correspondants. Les données reçues sont ensuite envoyées vers des nœuds debug afin de vérifier leur contenu, mais également vers les éléments `Temperature Mqtt` et `Humiditer Mqtt` du dashboard pour permettre leur affichage.

Pour commander la LED, un switch est utilisé dans le dashboard Node-RED. L'état sélectionné par l'utilisateur est transmis à un nœud MQTT Out configuré sur le topic `esp32/output`. Le switch envoie alors une valeur "true" lorsque la LED doit être allumée ou "false" lorsqu'elle doit être éteinte.

Cette valeur est publiée sur le broker Mosquitto, qui la transmet ensuite à l'ESP32. Comme l'ESP32 est abonné au topic `esp32/output`, sa fonction `callback()` est automatiquement appelée dès qu'un nouveau message est reçu. Le programme peut alors allumer ou éteindre la LED en fonction de la valeur contenue dans le message.

Un nœud debug 8 est également utilisé afin de visualiser la valeur envoyée par le switch et de vérifier que la commande est correctement transmise.

Ainsi, le PC joue ici le rôle de broker MQTT central grâce à Mosquitto. L'ESP32 publie les valeurs de température et d'humidité sur différents topics, Node-RED les récupère en s'y abonnant, puis Node-RED publie à son tour l'état du switch sur esp32/output afin de commander la LED de l'ESP32.

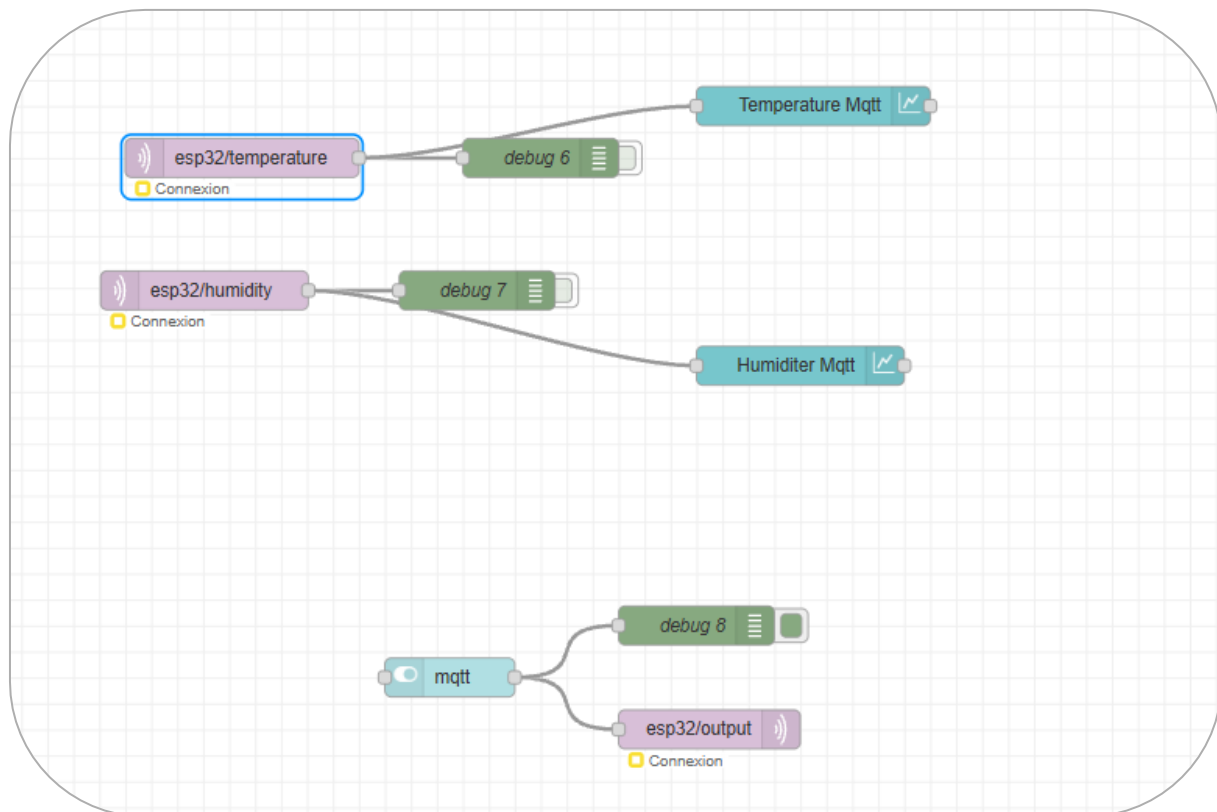


Figure 6 : flow node red mqtt

V. Résultat

Pour la partie résultat, la démonstration a été validée.

VI. Sources

Lors de ce projet différentes sources ont été utilisées pour ce mini-projet, voici les liens de ceux-ci.

- Sujet du TP : TP1_architecture_REST-MQTT_2026.pdf
- Documentation DHT11 fournie : DHT-11_datasheet.pdf
- Tutoriel ESP32 fourni : Tutorial_Getting_Started_ESP32.pdf

- PlatformIO - configuration platformio.ini :
<https://docs.platformio.org/en/stable/projectconf/index.html>
- PlatformIO - lib_deps :
https://docs.platformio.org/en/latest/projectconf/sections/env/options/library/lib_deps.html
- Node-RED : <https://nodered.org/>
- Node-RED Dashboard 3.6.6 : <https://flows.nodered.org/node/node-red-dashboard>
- Eclipse Mosquitto : <https://mosquitto.org/>
- Mosquitto mosquitto_sub : https://mosquitto.org/man/mosquitto_sub-1.html
- PubSubClient : <https://github.com/knolleary/pubsubclient>
- GitHub - ajouter un projet local : <https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github>